REWARDS — Rewards Program PWA

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.3

Campo	Descrição
Projeto	REWARDS — Rewards Program PWA (programa de fidelidade/cashback para varejo)
Gerente de Projeto	João Constantino Caetano
Gerente/Responsável de GCS-DevOps	João Constantino Caetano
Fábrica de Software / Squad	Projeto pessoal, desenvolvedor único (squad de 1 pessoa), com 1 revisor externo com acesso de escrita, exigido pela disciplina de GCS para aprovação de Pull Requests
Repositório principal	https://github.com/jjoaocc/rewards_program_pwa

Histórico de Revisões

Data	Versão	Descrição	Autor
10/08/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD).	Luiz Camargo
30/08/2026	1.1	Preenchimento das seções 1 a 3 com base no projeto real Rewards Program PWA.	João Constantino Caetano
03/09/2026	1.2	Preenchimento das seções 4 a 14 com base no projeto real Rewards Program PWA.	João Constantino Caetano
27/10/2026	1.3	Atualização das Seções 2, 3.2, 3.3, 4.2, 5.2, 7.1, 7.2, 8.1, 11 e 14 para refletir a	João Constantino Caetano

Data	Versão	Descrição	Autor
		proteção da branch main (Pull Request obrigatório, 1 aprovação, status checks obrigatórios, sem bypass para administradores), um colaborador com acesso de escrita como par de revisão, e os GitHub Environments staging/production, com aprovação manual obrigatória em produção.	

Sumário

(No Word, clique com o botão direito sobre esta área e escolha "Atualizar campo" — ou pressione Ctrl+A e F9 — para gerar o sumário automaticamente.)

TOC \h \o "1-3"

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe/individuo, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGC é destinado ao desenvolvedor único (indivíduo) responsável pelo desenvolvimento, operação e manutenção do **Rewards Program PWA**, um programa de fidelidade/cashback para varejo em produção real (https://rewards.strokes.dev.br), usado de fato pelo autor no dia a dia e mantido também como material de portfólio técnico. O sistema é composto por:

- um backend em FastAPI (Python 3.12), com arquitetura hexagonal (domain / ports / adapters / application / api);
- um frontend PWA instalável em React 19 + TypeScript + Vite + Tailwind CSS, com um painel administrativo separado (build multi-page) para disparo de notificações push;
- um banco de dados PostgreSQL, com schema versionado e migrações SQL manuais e sequenciais;
- os artefatos de containerização (Dockerfile de cada serviço, docker-compose.yml) e o pipeline de CI/CD (GitHub Actions).

Abrange o código-fonte de ambas as camadas, os scripts de banco de dados, os artefatos Docker e a definição do pipeline como código. Não há squad: todos os papéis de GCS/DevOps descritos neste plano (Seção 2) são exercidos pela mesma pessoa.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade

Termo	Significado
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM).
- Repositório do projeto: https://github.com/jjoaocc/rewards_program_pwa
- README.md do projeto, arquitetura hexagonal, decisões técnicas, guia de configuração e deploy
- Definição do pipeline de CI/CD: .github/workflows/ci.yml
- Documentação interativa da API (Swagger/OpenAPI): https://api.strokes.dev.br/rewards/docs

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação.** Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Prioriza o backlog, autoriza mudanças de maior impacto e acompanha indicadores de entrega (DORA Metrics).
Engenheiro DevOps / Release Engineer	Projeta e mantém os pipelines de CI/CD, define a estratégia de *branching* e versionamento, administra o registro de artefatos/imagens e a IaC.
Desenvolvedor(a)	Integra código com frequência na branch principal, escreve testes automatizados, revisa Pull Requests e é corresponsável pela qualidade do que é implantado.
SRE / Operações	Monitora ambientes de produção, define SLIs/SLOs, participa de

Papel	Responsabilidades
Site Reliability Engineering	postmortems e realiza rollback quando necessário. - *Service Level Indicators (SLIs) and Service Level Objectives (SLOs).*
QA / Automação de Testes	Mantém a suíte de testes automatizados (unitários, integração, aceitação) executada nos pipelines de CI e CD.
Segurança (DevSecOps)	Define políticas de análise estática (SAST), análise de dependências (SCA) e gestão de segredos integradas ao pipeline.
Comitê de Controle de Mudanças (CCM)	Atua apenas em mudanças de alto risco/impacto (ex.: mudanças em produção fora do fluxo padrão); mudanças de rotina seguem o fluxo automatizado de PR + pipeline.

No Rewards Program PWA, por se tratar de um projeto pessoal sem squad, todos os papéis acima são exercidos por João Constantino Caetano: define e prioriza o próprio backlog, projeta e mantém o pipeline de CI/CD (.github/workflows/ci.yml), escreve o código e os testes automatizados (181 no backend, 210 no frontend), administra a infraestrutura (VPS, Docker, Nginx Proxy Manager) e realiza o próprio deploy, mediante aprovação manual própria no GitHub Environment production (Seção 3.2). A revisão de código é a única responsabilidade que não pode ser exercida pelo próprio autor: desde 27/10/2026, o colaborador Tintomax tem acesso de escrita ao repositório especificamente para atuar como par de revisão de Pull Request, atendendo à regra de proteção da branch main (Seção 3.3), já que o GitHub não permite a autoaprovação da própria PR. Não existe um CCM formal: decisões de mudança de alto impacto (ex.: alterações de schema em produção) são avaliadas e aprovadas pelo próprio autor antes da execução, e o pipeline de CI (lint, tipos, segurança, testes) e a revisão do par funcionam como os mecanismos objetivos de controle antes de qualquer mudança chegar à produção.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas efetivamente adotadas pelo projeto:

Categoria	Ferramenta(s) adotada(s)	Finalidade
Controle de versão	Git + GitHub (github.com/jjoaocc/rewards_program_pwa)	Versionamento de código-fonte, IaC (docker-compose.yml) e do pipeline como código
Orquestração de CI/CD	GitHub Actions (runner hospedado do GitHub para CI; runner self-hosted na própria VPS para o deploy)	Automação de lint, tipos, segurança, testes e deploy a cada push em main
Empacotamento	Docker (multi-stage builds)	Geração das imagens do backend (Python) e do frontend (build Node → Nginx)
Registro de artefatos	Não há registry: as	Build local do artefato mais recente a cada

Categoria	Ferramenta(s) adotada(s)	Finalidade
	imagens são construídas diretamente na VPS de destino pelo runner self-hosted (docker compose build --no-cache)	deploy, sem publicação em registro externo
Infraestrutura como Código	Docker Compose (docker-compose.yml)	Definição declarativa dos serviços, redes (proxy-network, database-network) e variáveis de ambiente
Orquestração de containers	Docker Compose (containers isolados; sem Kubernetes/Swarm)	Execução dos containers rewards_backend e rewards_frontend na VPS
Gestão de segredos	Arquivo .env local (fora do versionamento, listado no .gitignore) + GitHub Actions Secrets	Armazenamento de credenciais (banco de dados, SECRET_KEY JWT, chaves VAPID, PUSH_ADMIN_SECRET)
Monitoramento e Observabilidade	Endpoint de health check HTTP (GET /health) verificado pós-deploy + logs via docker compose logs	Verificação de saúde pós-deploy e depuração manual
Gestão de dependências / SCA – Software Component Analisys	pip-audit (backend) e npm audit --audit-level=high (frontend)	Identificação de CVEs conhecidas nas dependências, a cada push
Análise estática e qualidade de código	Ruff + Mypy + Bandit (backend); ESLint + tsc (frontend)	Lint, checagem de tipos e análise de segurança estática (SAST) no pipeline de CI
Testes automatizados	Pytest + testcontainers, 181 testes, cobertura ~99% (backend); Vitest + React Testing Library, 210 testes (frontend)	Suíte de testes unitários (fakes em memória), de integração (Postgres real) e de contrato HTTP
Rastreamento de tarefas	GitHub Issues (quando aplicável)	Vinculação de commits a itens de trabalho, quando necessário

3.2 Ambientes e Infraestrutura

O projeto, por ser mantido por um único desenvolvedor, opera hoje com apenas dois ambientes reais (não há um ambiente formal de Homologação/Staging); a validação de qualidade antes da produção é feita pela suíte de testes automatizados executada no pipeline de CI, não por um ambiente separado.

Ambiente	Finalidade	Gatilho
Desenvolvimento	Máquina local do desenvolvedor: docker compose up -d --build (stack completa) ou modo dev do Vite (hot reload, localhost:5173) + uvicorn --reload para o backend.	Execução manual pelo desenvolvedor; sem pipeline associado.
Homologação / Staging	Não existe como ambiente de deploy separado; seu papel continua coberto pela suíte de testes automatizados (181 testes de backend + 210 de frontend, incluindo contrato HTTP via TestClient e Postgres real via testcontainers), agora rodada a cada Pull Request e a cada push em main. Um GitHub Environment staging foi criado em 27/10/2026 (exigência da disciplina de GCS) para fins de identificação e rastreabilidade de configuração, mas nenhum job do pipeline publica nele hoje; é uma lacuna conhecida caso o projeto evolua para ter um ambiente real de homologação.	Automático, a cada Pull Request e a cada push em main (jobs backend-ci/frontend-ci).
Produção	VPS própria: containers rewards_backend (FastAPI, rede database-network) e rewards_frontend (Nginx) atrás de Nginx Proxy Manager (SSL via Let's Encrypt) e Cloudflare (DNS); PostgreSQL compartilhado em rede Docker isolada. Nenhum container expõe porta diretamente no host.	Aprovação automática do gate de CI (backend-ci e frontend-ci), seguida de aprovação manual obrigatória no GitHub Environment production (revisor obrigatório: o próprio autor), antes do job deploy rodar no runner self-hosted na própria VPS.

3.3 Estratégia de Branching e Versionamento

Desde 27/10/2026, a branch main é protegida no GitHub: toda mudança passa obrigatoriamente por Pull Request, com no mínimo 1 aprovação de um revisor com acesso de escrita (Seção 2) e os dois status checks de CI (backend-ci e frontend-ci) obrigatoriamente verdes antes do merge; a opção enforce_admins está ativa, ou seja, nem o próprio autor, como administrador do repositório, pode contornar a regra com push direto. O workflow de CI (.github/workflows/ci.yml) dispara tanto em pull_request quanto em push para main, para que os status checks sempre tenham como rodar numa PR; o job deploy só executa no evento de push (pós-merge), nunca durante a revisão da PR.

- main, única branch de longa duração; sempre implantável; protegida por regra de proteção de branch (PR obrigatório, 1 aprovação, status checks obrigatórios, sem bypass para administradores); todo merge dispara o pipeline completo (CI + deploy condicionado ao CI verde e à aprovação manual no Environment production, Seção 3.2).
- Branches de feature, nomeadas segundo a convenção feature/<issue>-<descrição-curta> (Seção 4.2, ex.: feature/1-ci-environment-production), agora obrigatórias para qualquer mudança: a branch main não aceita mais push

direto, então toda mudança passa por uma branch de feature + Pull Request + revisão.
- Não há branches release/* ou hotfix/* em uso: correções emergenciais seguem o mesmo fluxo de Pull Request das demais mudanças, protegidas pela mesma regra de proteção de branch.

Versionamento (estado atual e recomendação): o projeto ainda não adota tags de versão SemVer nem Conventional Commits de forma sistemática (o campo version do package.json do frontend permanece em "0.0.0" e não há tags publicadas no repositório). Como evolução recomendada do processo, especialmente se o projeto ganhar colaboradores, sugere-se adotar Conventional Commits (feat, fix, chore, docs...) e tags SemVer a cada deploy em produção, permitindo gerar changelog automaticamente a partir do histórico de commits.

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, em um contexto DevOps os seguintes artefatos também são itens de configuração formais, versionados no mesmo repositório (ou repositórios compartilhados):

- Código-fonte da aplicação e dos testes automatizados
- Definições de pipeline como código (ex.: .github/workflows, Jenkinsfile, .gitlab-ci.yml)
- Dockerfiles e manifests de containers
- Scripts de Infraestrutura como Código (Terraform, Ansible, Helm charts, manifests Kubernetes)
- Arquivos de configuração por ambiente (.env.example como template; o .env real nunca é versionado, listado no .gitignore)
- Scripts de migração de banco de dados (database/migrations/*.sql, sequenciais e manuais) e o schema/seed base (database/schema/init.sql, database/seeds/seed_data.sql)
- Documentação técnica (README.md, com uma seção dedicada de Decisões Técnicas que cumpre, na prática, o papel de ADRs, Architecture Decision Records)
- Este próprio Plano de Gerenciamento de Configuração (docs/Plano_Ger_Conf_DevOps_Joao_Constantino_Caetano.docx), versionado no repositório como item de configuração formal

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch	feature/<issue>-<descrição-curta>, obrigatória para toda mudança desde a proteção da branch main (Seção 3.3)	feature/1-ci-environment-production
Commit	Mensagem descritiva em português; frequentemente no formato tipo: descrição (informal, não é Conventional	docs: prepara README para repositorio publico

Item	Convenção	Exemplo
	Commits estrito)	
Migração de banco	NNN_descricao_curta.sql, numeração sequencial de 3 dígitos	006_add_push_campaigns.sql
Variável de ambiente	UPPER_SNAKE_CASE	VAPID_PUBLIC_KEY
Imagem de container	Nome do serviço no docker-compose.yml, sem tag/versão nem publicação em registro	rewards_backend, rewards_frontend
Tag de release	Não utilizada hoje (ver 3.3); recomendação futura: SemVer vX.Y.Z	— (não adotado)

4.3 Estrutura do Repositório

Repositório único (monorepo), dividido por camada:

- backend/app/ — código FastAPI em arquitetura hexagonal: domain/ (entidades puras), ports/ (Protocols/contratos), adapters/db/ (implementação SQLAlchemy dos ports), application/ (casos de uso), api/v1/ (routers HTTP), core/ (config, JWT/segurança, rate limiting, push)
- backend/tests/ — unit/application/ (casos de uso com fakes em memória), integration/adapters/ (Postgres real via testcontainers), fakes/ e os testes de contrato HTTP (test_*.py)
- frontend/src/ — views/ (uma tela por rota), hooks/ (um hook por recurso da API, validado em runtime com zod), lib/ (cliente HTTP com porta injetável), contexts/, components/, types/, e admin/ (app React separado do painel administrativo, entry point próprio no Vite)
- database/ — schema/init.sql (schema base), seeds/seed_data.sql (dados mock), migrations/ (scripts SQL sequenciais e manuais, sem Alembic)
- .github/workflows/ci.yml — definição do pipeline de CI/CD como código (Seções 5 e 6)
- docker-compose.yml na raiz + um Dockerfile por serviço (backend/Dockerfile, frontend/Dockerfile)
- docs/ — este Plano de Gerenciamento de Configuração (.docx)
- .env.example — template de variáveis de ambiente (o .env real está no .gitignore e nunca é commitado)

5. Integração Contínua (CI)

Todo push na branch main dispara automaticamente o pipeline de CI (o projeto não usa Pull Requests — ver 5.2). Nenhuma mudança chega ao job de deploy (Seção 6) sem que os dois jobs de CI abaixo terminem com sucesso.

5.1 Etapas do Pipeline de CI

Dois jobs independentes rodam em paralelo a cada push (runs-on: ubuntu-latest), ambos condição para o deploy:

Job backend-ci

1. Checkout do código.
2. Configuração do Python 3.12.
3. Instalação das dependências (requirements.txt + requirements-dev.txt).
4. Lint (Ruff).
5. Checagem de tipos estática (Mypy).
6. Análise de segurança estática — SAST (Bandit).
7. Auditoria de CVEs conhecidas nas dependências — SCA (pip-audit).
8. Testes automatizados com cobertura (Pytest, 181 testes, ~99% de cobertura).

Job frontend-ci

1. Checkout do código.
2. Configuração do Node 22 (cache de npm).
3. Instalação das dependências (npm ci).
4. Auditoria de CVEs conhecidas nas dependências — SCA (npm audit --audit-level=high).
5. Lint (ESLint).
6. Testes automatizados (Vitest + React Testing Library, 210 testes).
7. Build de produção (tsc -b + vite build, valida os dois entry points: app e admin).

Não há uma etapa separada de empacotamento/publicação de imagem no CI: o build da imagem Docker só acontece no job de deploy (Seção 6.1), diretamente na VPS de destino.

5.2 Política de Pull Request e Revisão de Código

Desde 27/10/2026, toda mudança passa por Pull Request: a branch main é protegida no GitHub (Seção 3.3) e exige no mínimo 1 aprovação antes do merge, de um colaborador com acesso de escrita (Tintomax), já que o GitHub não permite a autoaprovação da própria PR. O merge só é liberado com os dois status checks de CI (backend-ci e frontend-ci) verdes; a opção enforce_admins impede que o próprio autor, mesmo como administrador do repositório, contorne essa regra com push direto, o que foi verificado na prática (git push direto recusado com o erro GH006: Protected branch update failed). Canal de comunicação e janela de resposta: como a revisão é feita por um colaborador externo e pontual, a comunicação ocorre pelos próprios comentários da Issue/Pull Request no GitHub, sem canal de chat dedicado; não há uma janela de resposta formalizada hoje, dado o baixo volume de PRs; recomenda-se adotar um prazo combinado (ex.: até 2 dias úteis) caso o ritmo de contribuições aumente.

5.3 Critérios de Qualidade (Quality Gate)

Critérios objetivos hoje aplicados (todos bloqueantes: qualquer falha impede o deploy):

- Backend: ruff check sem erros (regras E, F, I, UP, B, line-length 120); mypy app/ sem erros (check_untyped_defs, warn_redundant_casts, warn_unused_ignores ativos); bandit -r app/ sem achados; pip-audit sem CVEs conhecidas nas dependências; 100% dos 181 testes (Pytest) passando.

- Frontend: npm audit --audit-level=high sem vulnerabilidades high/critical; eslint sem erros; 100% dos 210 testes (Vitest) passando; npm run build (tsc -b + vite build) sem erros de tipo ou de build.
- Deploy: só é executado se backend-ci e frontend-ci passarem; a própria checagem de saúde pós-deploy (Seção 6.1) é o último critério, aplicado depois do deploy.

Lacuna conhecida: não há um limite mínimo de cobertura de testes configurado como gate no pipeline (a cobertura do backend é reportada por --cov-report=term-missing, mas o job não falha abaixo de um limiar; ex.: --cov-fail-under=90 seria a evolução natural).

6. Entrega e Implantação Contínua (CD)

Assim que os dois jobs de CI (backend-ci e frontend-ci) terminam com sucesso, o job deploy é habilitado. Não há publicação prévia de artefato em registro (Seção 3.1): a imagem é construída no momento do próprio deploy, direto na VPS de produção, não existe promoção entre ambientes porque só existe um ambiente de destino (Seção 3.2).

6.1 Etapas do Pipeline de CD

O job deploy roda em um runner self-hosted registrado na própria VPS (runs-on: [self-hosted, vps]):

1. git pull origin main no diretório do projeto na VPS (/opt/containers/rewards).
2. docker compose build --no-cache — reconstrói as imagens do backend e do frontend a partir do código recém-puxado.
3. docker compose up -d — recria os containers rewards_backend e rewards_frontend.
4. Reload do Nginx Proxy Manager (docker exec nginx-proxy-manager nginx -s reload) — necessário porque o NPM cacheia o IP Docker anterior do container, que muda a cada recriação.
5. Checagem de saúde pós-deploy: até 15 tentativas (a cada 4s) de curl em /health do backend e na raiz do frontend; se nenhuma responder no tempo, o workflow imprime os últimos 80 logs e falha.
6. Limpeza de imagens Docker órfãs (docker image prune -f), executada sempre (if: always()), mesmo se a checagem de saúde falhar.

6.2 Estratégias de Implantação

Estratégia	Descrição
Recreate (estratégia atual)	docker compose build --no-cache seguido de up -d recria o container do zero; há uma janela breve de indisponibilidade entre o container antigo ser derrubado e o novo ficar pronto, mitigada pela checagem de saúde pós-deploy (6.1), mas sem rollback automático (6.3).
Rolling update / Blue-Green / Canary Release (não implementadas)	Recomendadas como evolução caso o sistema exija indisponibilidade zero ou releases graduais; hoje não se justificam pelo baixo tráfego e criticidade do projeto (uso pessoal/portfólio).

Estratégia	Descrição
Feature Flags (não implementadas)	Nenhuma funcionalidade é hoje controlada por flag em tempo de execução; ativações dependem diretamente do deploy do código.

6.3 Rollback e Recuperação

Não há rollback automatizado hoje. Se a checagem de saúde pós-deploy (6.1) falhar, o workflow falha e os últimos logs ficam visíveis no histórico do GitHub Actions, mas a versão com problema pode já estar servindo tráfego (o job não reverte automaticamente para a imagem anterior). A recuperação atual é manual: git revert/checkout do commit anterior em main + novo push, o que dispara um novo ciclo completo de CI/CD.

Como evolução recomendada: manter a imagem anterior versionada (tag) para permitir docker compose up com a tag anterior sem reconstrução, e considerar rollback automático (voltar à imagem anterior) caso a checagem de saúde pós-deploy falhe.

7. Infraestrutura como Código (IaC)

A infraestrutura do ambiente de Produção (único ambiente gerenciado por código — Seção 3.2) é descrita via Docker Compose, versionado no mesmo repositório da aplicação. Não se usa uma ferramenta de provisionamento em nuvem (Terraform/Ansible/Pulumi): a VPS em si e os serviços compartilhados (PostgreSQL, Nginx Proxy Manager, redes Docker) são provisionados manualmente, uma única vez, fora do repositório.

7.1 Versionamento de Infraestrutura

- docker-compose.yml e os Dockerfile de cada serviço definem os containers da aplicação e são versionados junto do código-fonte.
- Não há uma etapa de "plan" prévio (como em Terraform): mudanças na infraestrutura de containers seguem o mesmo fluxo de Pull Request + revisão + pipeline de CI/CD usado para o restante do código (Seções 3.3 e 5.2), sem uma etapa de revisão de infraestrutura separada.
- Não há state file a gerenciar — Docker Compose não mantém estado remoto como Terraform; a infraestrutura de containers reflete diretamente o que está declarado no docker-compose.yml no momento do deploy.
- As redes Docker (proxy-network, database-network) e o PostgreSQL compartilhado são provisionados manualmente uma única vez na VPS (docker network create ...), documentados no README.md mas fora do escopo versionado deste repositório.

7.2 Ambientes Efêmeros

O projeto não utiliza ambientes efêmeros/review apps: mesmo com o fluxo de Pull Request agora obrigatório (Seção 5.2), a validação de cada mudança antes do merge continua feita pela suíte de testes automatizados do CI (Seção 5.1), não por um ambiente dedicado por PR. Como evolução recomendada, ambientes efêmeros por PR (ex.: uma instância Docker Compose dedicada, atrás de um subdomínio temporário)

poderiam complementar essa validação, reduzindo ainda mais o risco de regressões não cobertas por teste.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo real do projeto: idealização/planejamento pelo próprio autor, registrado como GitHub Issue (Seção 3.1) → branch de feature feature/<issue>-<descrição-curta> (Seções 3.3/4.2) → Pull Request → aprovação de 1 revisor com acesso de escrita (Seção 5.2) → pipeline de CI (Seção 5) → merge → deploy automático em produção, mediante aprovação manual no GitHub Environment production (Seções 3.2/6). A abertura de Issue e Pull Request é obrigatória desde a proteção da branch main (27/10/2026); o histórico de Issues, Pull Requests e execuções do pipeline no GitHub Actions (CI + deploy + checagem de saúde) constitui, em conjunto, o registro rastreável de cada mudança.

8.2 Comitê de Controle de Mudança (CCM)

Como o projeto é mantido por um único desenvolvedor, não existe um CCM formal, nem há composição a definir: mudanças de alto impacto ou risco elevado (ex.: alteração de schema em produção, rotação das chaves VAPID, mudança na estratégia de autenticação) são decididas e executadas pelo próprio autor, com testagem local redobrada antes do push, já que não há uma segunda pessoa para revisar. Em situações rotineiras, o próprio pipeline de CI (lint, tipos, segurança, testes) cumpre o papel de controle objetivo que, em uma squad maior, seria feito pela revisão de PR + CCM.

Caso o projeto ganhe colaboradores, recomenda-se instituir um CCM leve (o(s) próprio(s) mantenedor(es)) acionado apenas para: mudanças de schema de banco em produção, rotação de segredos/chaves (VAPID, SECRET_KEY, PUSH_ADMIN_SECRET), ou qualquer exceção ao fluxo padrão (ex.: hotfix emergencial aplicado direto em produção sem passar pela suíte completa de testes).

9. Baselines e Releases

9.1 Geração Automática de Baseline

Diferente de modelos tradicionais, em que a baseline é criada manualmente em marcos do projeto, o ideal é que cada implantação bem-sucedida em produção gere automaticamente uma baseline (tag de versão associada ao commit e ao estado da infraestrutura). No estado atual do Rewards Program PWA essa geração automática ainda não está implementada (consistente com a ausência de tags SemVer descrita na Seção 3.3): cada deploy bem-sucedido constitui, na prática, uma baseline implícita — o commit em main no momento do deploy, rastreável pelo histórico de commits e pelo run correspondente no GitHub Actions — mas sem uma tag Git formal associada. Como evolução recomendada, o job deploy poderia criar automaticamente uma tag SemVer a cada execução bem-sucedida.

9.2 Notas de Versão (Release Notes)

Pelo mesmo motivo, não há geração de notas de release hoje: sem tags e sem Conventional Commits sistemáticos (Seção 3.3), não existe uma fonte estruturada para gerá-las automaticamente. Como evolução recomendada, adotar Conventional Commits de forma consistente permitiria gerar cada release note automaticamente (ex.: com ferramentas como release-please ou semantic-release) a partir do histórico de commits desde a última tag.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Diferente de uma stack de observabilidade completa (Prometheus/Grafana, alertas automáticos — listada apenas como referência na Seção 3.1), o monitoramento hoje se resume a: (1) a checagem de saúde pós-deploy do próprio pipeline de CD (Seção 6.1), que falha visivelmente no GitHub Actions caso o backend ou o frontend não respondam logo após a implantação; e (2) inspeção manual de logs via docker compose logs quando necessário. Não há alertas proativos: se o serviço cair horas depois de um deploy que passou na checagem de saúde, não existe notificação automática. Como evolução recomendada: um serviço de uptime monitoring (ex.: UptimeRobot, Better Uptime) apontando para /health e para o frontend, com alerta por e-mail/push em caso de indisponibilidade.

10.2 Métricas DORA

As métricas abaixo não são medidas formalmente hoje (sem dashboard dedicado), mas todas são deriváveis dos dados já existentes — histórico de commits do Git e histórico de execuções do GitHub Actions:

Métrica	Descrição	Como medir neste projeto
Deployment Frequency	Frequência com que o código é implantado em produção.	Contagem de execuções bem-sucedidas do job deploy no histórico do GitHub Actions.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.	Diferença entre o timestamp do commit e o timestamp de conclusão do run do Actions correspondente.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.	Proporção de runs do workflow em que o CI ou o job deploy (checagem de saúde) falham, sobre o total de pushes em main.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.	Tempo entre um deploy que falhou a checagem de saúde e o próximo push que corrige o problema (apurado manualmente hoje, olhando o histórico de Actions).

10.3 Relatórios e Auditorias

O histórico de execuções do GitHub Actions (CI + deploy + checagem de saúde) e o histórico de commits do Git constituem, por si, o principal insumo de auditoria de GCS deste projeto, dispensando relatórios manuais na maior parte dos casos.

11. Marcos

- [Concluído] Implantação do pipeline de CI — lint, checagem de tipos, análise de segurança estática e auditoria de dependências para backend e frontend (Seção 5).
- [Concluído] Suíte de testes automatizados abrangente (181 testes de backend, cobertura ~99%; 210 de frontend) como pré-requisito de todo deploy.
- [Concluído] Implantação do pipeline de CD — deploy automático em produção via runner self-hosted, com checagem de saúde pós-deploy (Seção 6).
- [Concluído] Publicação do repositório como código aberto (portfólio técnico).
- [Concluído] Proteção da branch main (Pull Request obrigatório, 1 aprovação, status checks obrigatórios, sem bypass para administradores), par de revisão com acesso de escrita (Tintomax) e GitHub Environments staging/production, com aprovação manual obrigatória em produção (27/10/2026).
- [Planejado] Adoção de SemVer + Conventional Commits, com geração automática de tag/release note a cada deploy (Seção 9).
- [Planejado] Rollback automatizado em caso de falha na checagem de saúde pós-deploy (Seção 6.3).
- [Planejado] Monitoramento proativo (uptime + alertas), além da checagem pontual pós-deploy (Seção 10.1).
- [Planejado] Cobertura mínima de testes como gate explícito no CI (Seção 5.3).

12. Treinamento e Recursos

Por se tratar de um projeto individual, não há necessidade de treinamento de equipe; o principal "recurso" de GCS/DevOps é a própria documentação mantida no repositório:

- README.md — guia completo de arquitetura, configuração de ambiente, banco de dados, testes, CI/CD e deploy; ponto de entrada para qualquer pessoa (inclusive o próprio autor, no futuro) retomar o projeto.
- A seção Decisões Técnicas do README.md, que registra o raciocínio por trás de escolhas como a arquitetura hexagonal, o uso de zod no frontend, a ausência de Alembic e o runner self-hosted — cumprindo, na prática, o papel de ADRs.
- Todas as ferramentas usadas (Ruff, Mypy, Bandit, pip-audit, ESLint, Vitest, Docker, GitHub Actions) são de código aberto, com documentação oficial gratuita, sem necessidade de licenciamento ou treinamento pago.

Caso o projeto ganhe colaboradores, o README.md já serve como onboarding técnico; a lacuna a suprir seria um guia de contribuição (CONTRIBUTING.md) cobrindo o fluxo de Pull Request recomendado (ainda não existente, Seção 5.2).

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas e serviços de terceiros integrados ao produto também são itens de configuração e devem ser versionados explicitamente (arquivo de dependências travado — lockfile).

- Análise automática de vulnerabilidades em dependências (SCA) integrada ao pipeline de CI: pip-audit (backend, requirements.txt) e npm audit --audit-level=high (frontend, package-lock.json), a cada push (Seção 3.1/5.1).
- Atualizações de dependências hoje são manuais (não há Dependabot/Renovate configurado no repositório); toda atualização passa pelo mesmo pipeline de CI antes de chegar a produção.

Não há subcontratados nem fornecedores de software externos: todo o código é desenvolvido pelo próprio autor. As únicas dependências de terceiros são as bibliotecas open-source travadas em requirements.txt (backend) e package-lock.json (frontend). Os únicos serviços externos utilizados são de infraestrutura, não de software sob encomenda — Cloudflare (DNS) e Let's Encrypt via Nginx Proxy Manager (certificados SSL) — nenhum deles integra código ao pipeline, por isso não se aplica um processo de incorporação de software de terceiros além da auditoria de dependências já descrita acima.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo real, do commit à produção:

Etapa	Descrição	Gatilho
CI — backend-ci	Ruff, Mypy, Bandit, pip-audit, Pytest (181 testes).	Pull Request para main e push em main (após merge)
CI — frontend-ci	npm audit, ESLint, Vitest (210 testes), build (tsc + vite).	Pull Request para main e push em main (após merge), em paralelo ao anterior
Revisão (Pull Request)	1 aprovação de revisor com acesso de escrita (obrigatória); branch main protegida, sem bypass para administradores (enforce_admins).	CI verde (backend-ci e frontend-ci)
Deploy	git pull + docker compose build --no-cache + up -d + reload do Nginx Proxy Manager.	Merge em main após aprovação de PR; aprovação manual adicional no GitHub Environment production (revisor: o próprio autor)

Etapa	Descrição	Gatilho
Checagem de saúde	Até 15 tentativas de curl em /health (backend) e na raiz (frontend); falha o workflow se não responder.	Após o deploy
Limpeza	docker image prune -f (remove imagens Docker órfãs).	Sempre, ao final do job deploy (mesmo se a etapa anterior falhar)

Nota: *este template deve ser adaptado ao contexto real do projeto (ferramentas, criticidade do sistema e maturidade da equipe em DevOps), mantendo os quatro pilares da GCS — identificação, controle de versão, controle de mudanças e auditoria/status — agora suportados por automação.*